

Polymorphism and Composition

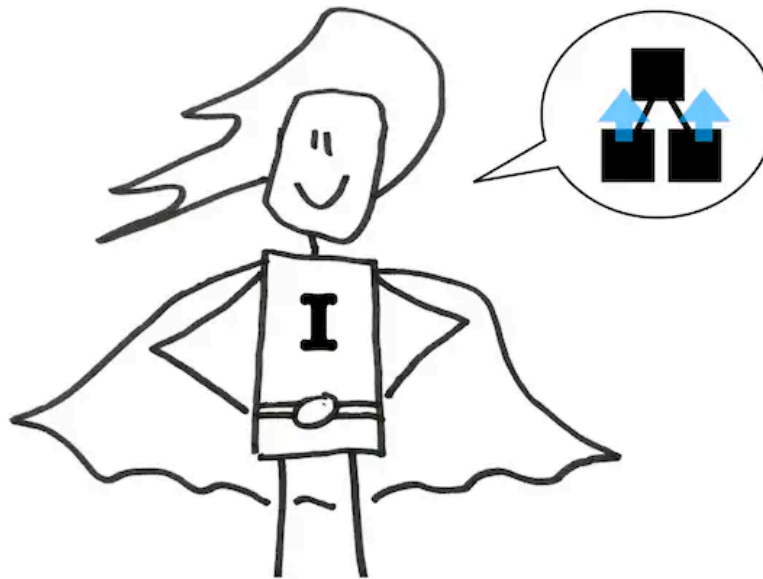
Detecting Duplication

- When we compare the UMLs of the two report classes, we see duplications.

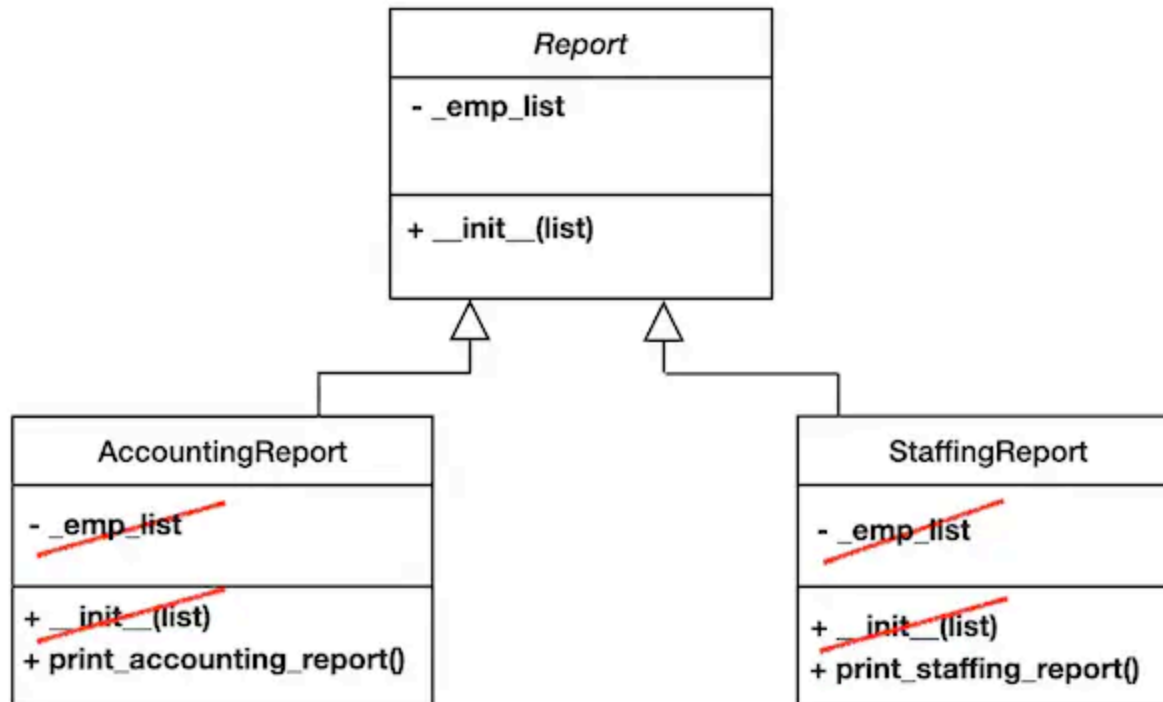


| Inheritance as the Solution

- We already know the solution: inheritance.



Refactored Design and Code



```
class Report:
    def __init__(self, emp_list):
        self._emp_list = emp_list

class AccountingReport(Report):
    def print_accounting_report(self):
        print("Accounting")
        print("=====")
        for e in self._emp_list:
            print(f"{e.get_full_name()}, ${e.salary}")

class StaffingReport(Report):
    def print_staffing_report(self):
        print("Staffing")
        print("=====")
        for e in self._emp_list:
            print(f"{e.get_full_name()}, {e.job_title}")
```

| Refactor the main.py

- We refactor the main.py to have the reports list and a loop to invoke the print methods.
- Sub-classes have **different method names**, so we should check.

```
from employee import Manager, Attendant, Cook, Mechanic
from reporting import AccountingReport, StaffingReport

employees = create_employees() # helper defined in main.py; existing Employee objects
reports = [
    AccountingReport(employees),
    StaffingReport(employees),
]

# ?????? If statement again?

for report in reports:
    if isinstance(report, AccountingReport): # if/else
        report.print_accounting_report()
    else:
        report.print_staffing_report()
```

Code Smell: Type-Based Dispatch

- `isinstance()` itself isn't a code smell — it's genuinely useful for things like validation or parsing.
- The smell here is **branching on an object's type to decide what to do**: every time we add a new Report subtype, we must come back and edit this if/else chain (an OCP violation).

```
for report in reports:  
    if isinstance(report, AccountingReport): # if/else  
        report.print_accounting_report()  
    else:  
        report.print_staffing_report()
```

Polymorphism as the Solution

- We can use **Polymorphism** to solve this issue.
- Polymorphism means different objects respond to the **same method call** with type-specific behavior — not just that different subclasses happen to behave differently.

- When different objects have the same method name, we can invoke the correct method using Polymorphism.

Tool box

✓ **Objects & Classes**

✓ **Inheritance**

✓ **Encapsulation**

✓ **Polymorphism**

Composition

How Polymorphism Works

- We can use the same method name for the Report classes.

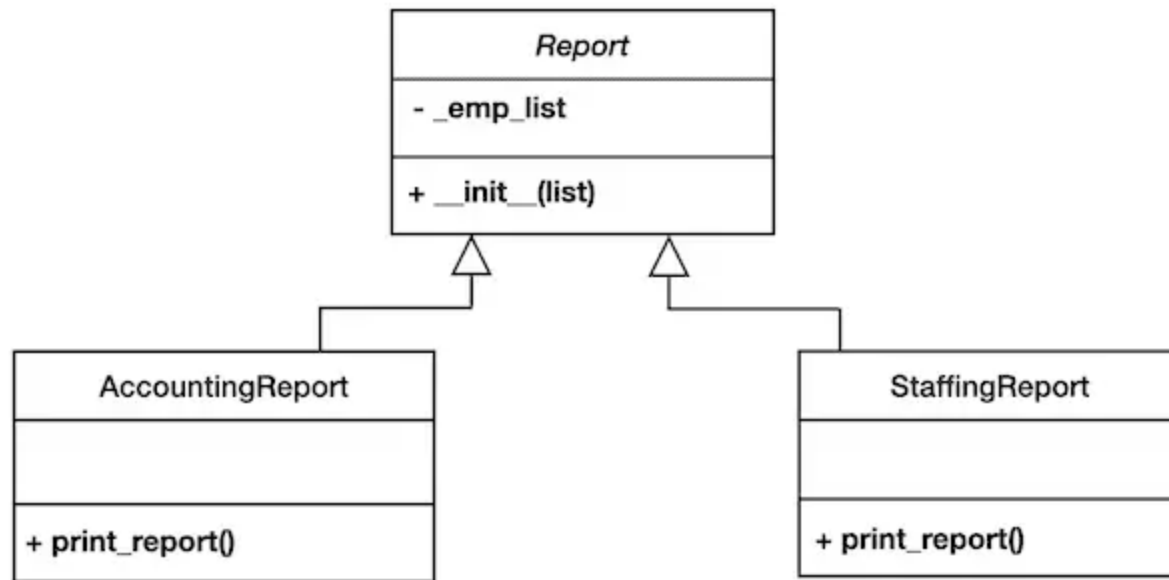
```
for r in reports:  
    if type(r) is AccountingReport:  
        r.print_accounting_report()  
    elif type(r) is StaffingReport:  
        r.print_staffing_report()  
    elif type(r) is AllowanceReport:  
        r.print_allowance_report()  
    elif type(r) is VacationReport:  
        r.print_vacation_report()
```



```
for r in reports:  
    r.print_report()
```

Refactoring the UML

- The first step is to refactor the UML.



Clean Implementation

- Then, we can remove the if statement in the main.py.
- The loop uses polymorphism through the shared `Report` superclass.
- Python can also do this with duck typing, as long as each object has a `print_report()` method.

```
# reporting.py

class AccountingReport(Report):
    def print_report(self):
        print("Accounting"); print("=====")
        for e in self._emp_list:
            print(f"{e.get_full_name()}, ${e.salary}")

class StaffingReport(Report):
    def print_report(self):
        print("Staffing"); print("=====")
        for e in self._emp_list:
            print(f"{e.get_full_name()}, {e.job_title}")
```

- The main function invokes the `print_report` method from each object.
- No if/else statement is needed.

```
# main.py
reports = [AccountingReport(employees), StaffingReport(employees)]
for report in reports:
    report.print_report() # Polymorphism in action!
```

| Open-Closed Principle

- Polymorphism has a close relationship with the OCP (Open-Closed Principle) of SOLID.
- Polymorphism can support OCP: new report types can be added without modifying the report-processing loop.

Lessons Learned

- When we use if/else statements with type checking, it is a **code smell**.
- Instead, we use Polymorphism: the **same method name** and invoke the method on the objects.

IMPORTANT

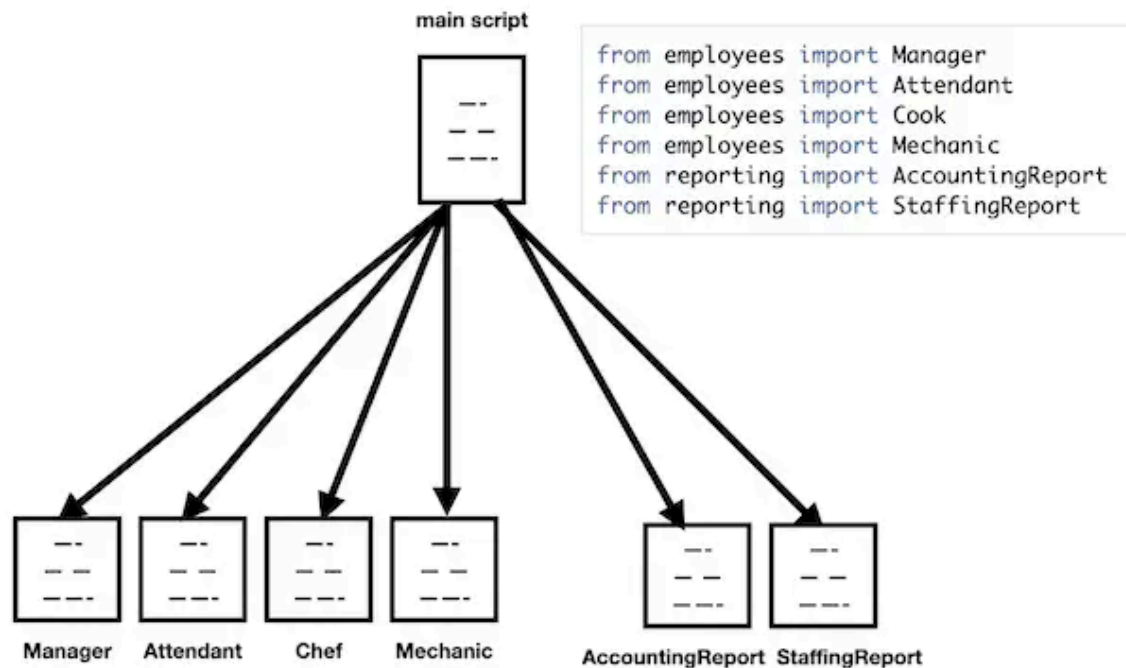
- **Key Benefits of Polymorphism**
 - No type checking
 - Easy to add new report types
 - Simpler, cleaner code
 - Better OCP support

Reporting Relationships and SRP (Single Responsibility Principle)

Software Design with APIE

- With the APIE principle guidelines, our software is well-modularized with clear interfaces.
- The dependency is well-designed and defined; we can use UML to clarify the dependencies with arrows.

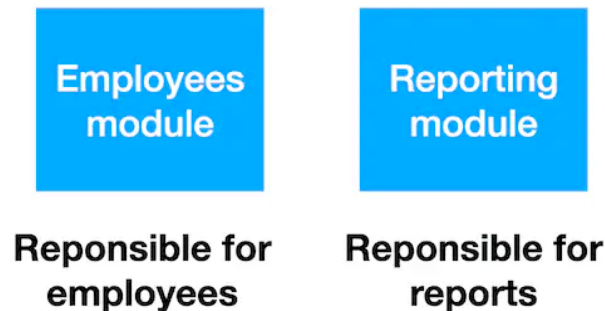
- This is a simplified view of our software design.



| Single Responsibility Principle (SRP)

- The **Employees module** is responsible only for employees.
- Likewise, **reporting module** is only responsible for reporting.

- This idea is called the SRP (Single Responsibility Principle) of SOLID.



SRP Key Points

- SRP means a module should have **one reason to change** — not that it may only touch one entity.
- A module can handle multiple related entities and still satisfy SRP, as long as they all change for the same reason.

Problem indicators

~~Duplicate code~~

~~Coupling~~

~~No Single Responsibility~~

~~if/else~~

- Remember: **S** in SRP stands for **Single (reason to change)**, not Simple.

New Feature: Working Hours

- Mr.Star wants to track working hours.
- There are two shifts: some employees start from 08:00 am to 2:00 pm (morning shift), while others start from noon to 8:00 pm (afternoon shift).

- Mr. Star also wants a new report:
ScheduleReport.

FIRST NAME	LAST NAME	SHIFT
Vera	Schmidt	8:00 - 14:00
Chuck	Norris	8:00 - 14:00
Samantha	Carrington	12:00 - 20:00
Roberto	Jacketti	8:00 - 14:00
Dave	Dreißig	8:00 - 14:00
Tina	River	8:00 - 14:00
Ringo	Rama	12:00 - 20:00
Chuck	Rainey	12:00 - 20:00

| Requirements Version 6

We have new requirements.

Epic requirement:

As an "employer,"

I want to "generate reports,"

so that "I can manage my employees."

Sub requirement 1:

As an "account manager,"

I want to "have an Accounting report including first name, last name, and salary,"
so that "I can track monthly salary payments."

Sub requirement 2:

As a "staff manager,"

I want to "have a Staffing report including first name, last name, and job title,"
so that "I can track my staff."

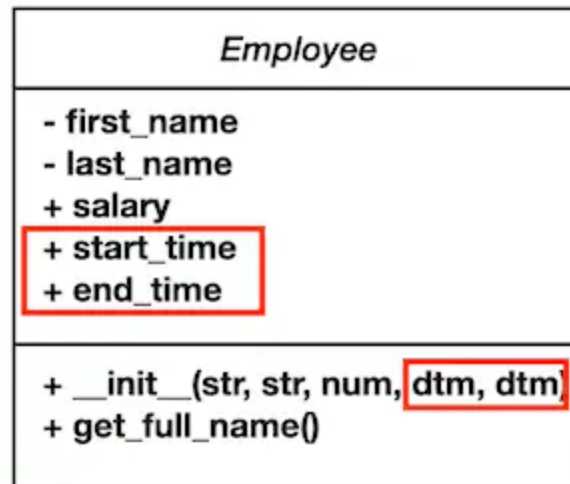
Sub requirement 3:

As a "CEO,"

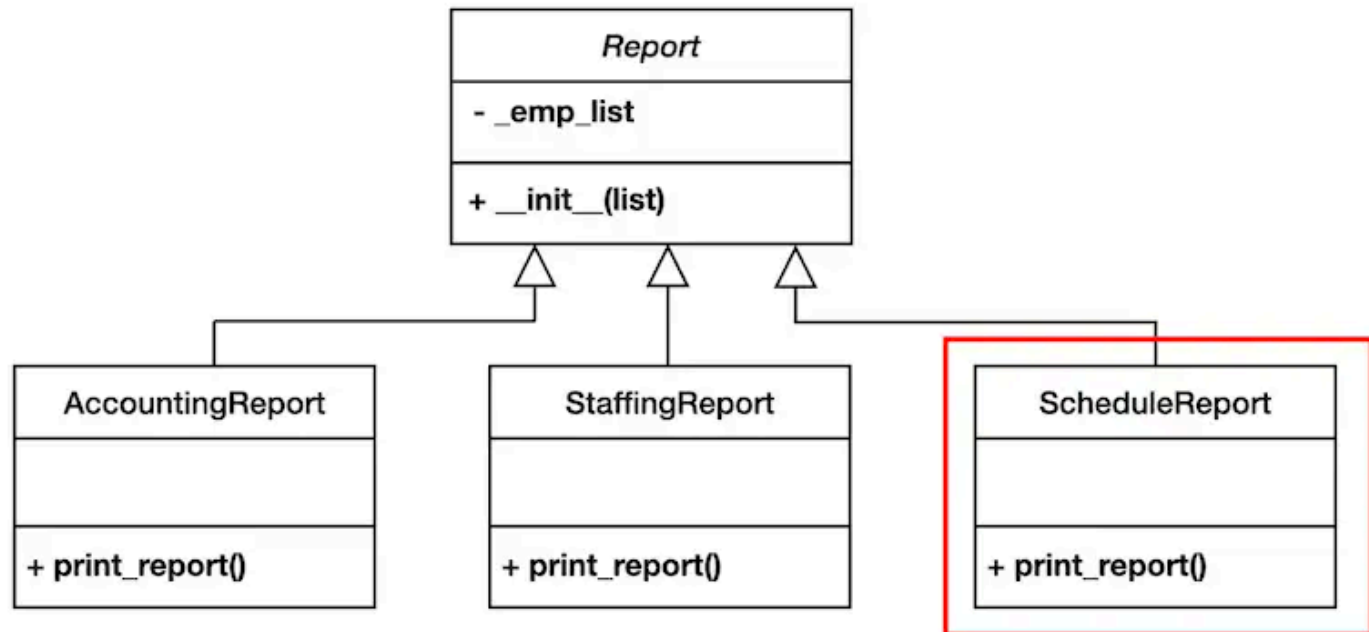
I want to "have a Schedule report including start time and end time,"
so that "I can track my staff shift."

Design Changes

- We need to refactor the UML design, Employee, to include start and end time.



- For the Reporting class, we can extend the Reporting class.



Implementation

We refactor the Employee class and create the ScheduleReport class.

```
class Employee:
    def __init__(self, last_name, first_name,
                  salary, start_time, end_time):
        self._first_name = first_name
        self._last_name = last_name
        self.salary = salary
        self.start_time = start_time
        self.end_time = end_time
```

```
class ScheduleReport(Report):
    def print_report(self):
        print("Schedule"); print("=====")
        for e in self._emp_list:
            print(f"{e.get_full_name()},
                {e.start_time:%H:%M} to {e.end_time:%H:%M}")
```

IMPORTANT

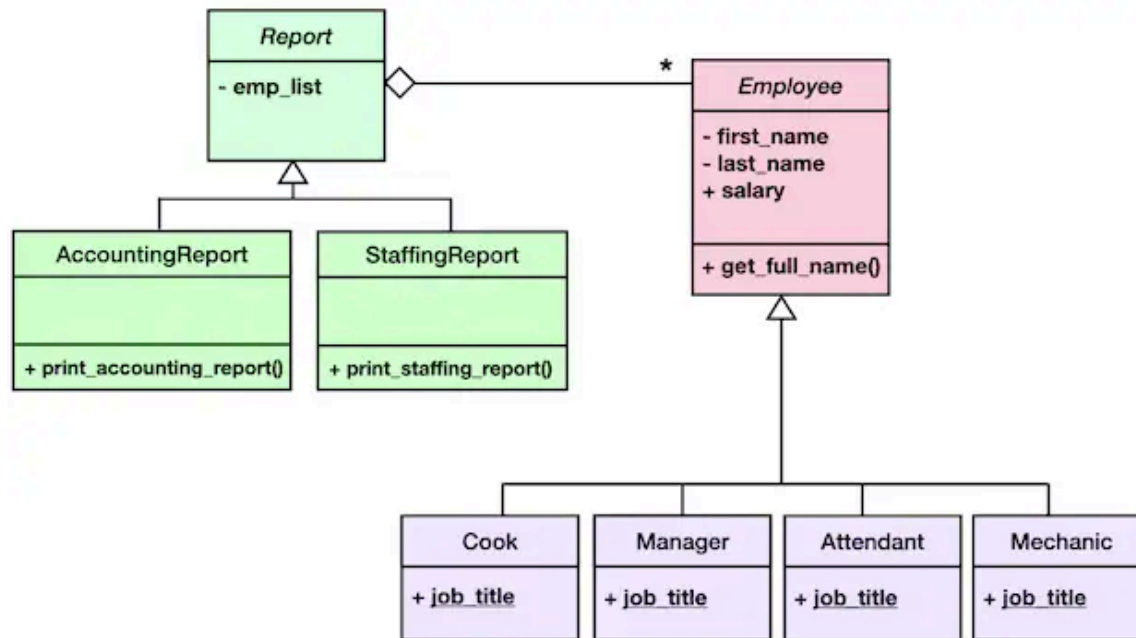
Aggregation Relationship

- The Report classes need a collection of Employees to do their work.
- We can model this as an **aggregation**: Report has a group of Employees, but does not own their lifecycle — the same Employee list is created elsewhere and shared across multiple reports.

UML

- The UML uses an **empty diamond** for aggregation.
- The diamond is placed on the side that contains the other object without owning its lifecycle.
- In this design, the diamond is on the **Report** side.

Software Design



- The * on Employee means a Report can hold multiple objects.
- The report owns high-level abstraction (Employee), not a low-level class.
- The report itself is a high-level abstraction with multiple concrete classes.

The direction of a diamond

- In our design, the focus is on the Report.
- The Report needs the Employees for its reporting, not the other way round.
- So, the diamond (aggregation) goes on the Report side, even though the Employees are shared and not exclusively owned by any one Report.

Aggregation and SRP

- Aggregation describes the relationship; SRP explains the design intent.
- **Employee** and **Report** change for different reasons.
- **Employee** changes for employee data; **Report** changes for reporting behavior.
- The empty diamond shows the structure, not the responsibility boundary.

Easy Extension

- The design is flexible.
- Adding a new report type requires only one new class and one registration line.
- The processing loop stays unchanged.

```

# main.py
import datetime

employees = [
    Manager("Schmidt", "Vera", 2000,
            datetime.time(8, 0), datetime.time(14, 0)),
    Attendant("Norris", "Chuck", 1800,
              datetime.time(12, 0), datetime.time(20, 0)),
    # ... more employees
]

reports = [
    AccountingReport(employees),
    StaffingReport(employees),
    ScheduleReport(employees) # <- Added !!!
]

for report in reports:
    report.print_report()
    print()

```

...

Staffing

=====

Vera,Schmidt, 08:00 to 14:00
Chuck,Norris, 08:00 to 14:00
Samantha,Carrington, 12:00 to 20:00
Roberto,Jacketti, 08:00 to 14:00
Dave,Dreißig, 08:00 to 14:00
Tina,River, 08:00 to 14:00
Ringo,Rama, 12:00 to 20:00
Chuck,Rainey, 12:00 to 20:00

Lessons Learned

- Using software design, we can manage complexity.
- Software design helps to detect code smells and refactor them.
- With good design, we can add new functionality with minimal code changes.

Explosion of Classes and Composition as the solution

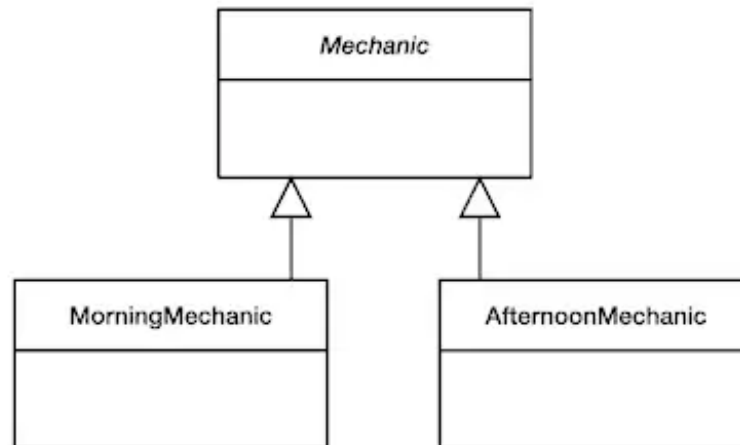
Code Smell: Duplication

- We have only two time shifts (morning and afternoon), so we **duplicate** the start time and end time.

```
employees = [  
    Manager("Vera", "Schmidt", 2000, datetime.time(8, 00), datetime.time(14, 00)),  
    Attendant("Chuck", "Norris", 1800, datetime.time(8, 00), datetime.time(14, 00)),  
    Attendant("Samantha", "Carrington", 1800, datetime.time(12, 00), datetime.time(20, 00)),  
    Cook("Roberto", "Jacketti", 2100, datetime.time(8, 00), datetime.time(14, 00)),  
    Mechanic("Dave", "Dreißig", 2200, datetime.time(8, 00), datetime.time(14, 00)),  
    Mechanic("Tina", "River", 2300, datetime.time(8, 00), datetime.time(14, 00)),  
    Mechanic("Ringo", "Rama", 1900, datetime.time(12, 00), datetime.time(20, 00)),  
    Mechanic("Chuck", "Rainey", 1800, datetime.time(12, 00), datetime.time(20, 00)),  
]
```

| Inheritance?

- Maybe, we can extend the Mechanics class to have MorningMechanic and AfternoonMechanic.



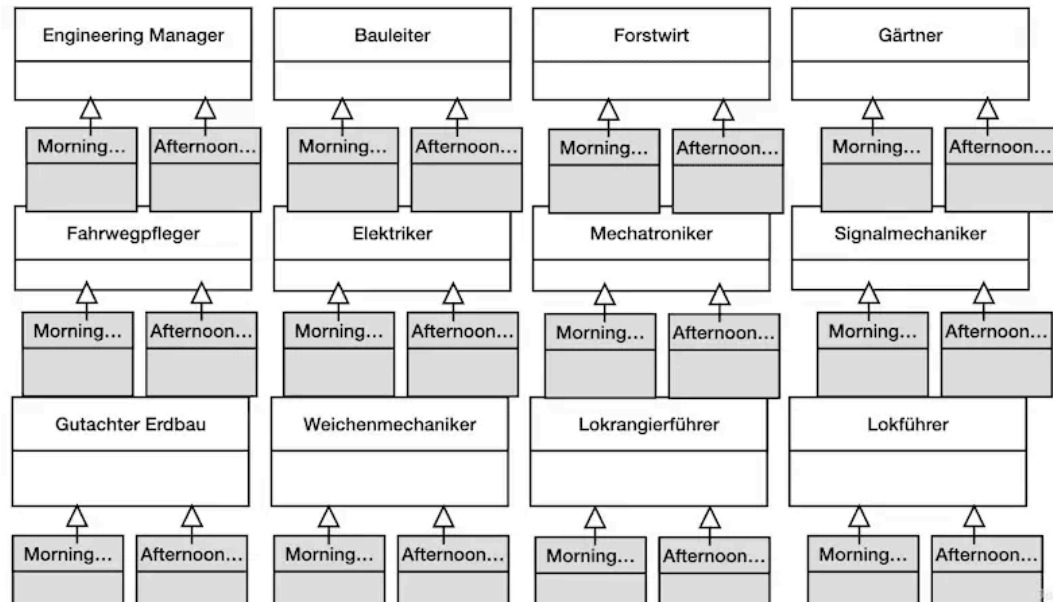
The Scaling Problem

- However, what if we have tens of job titles?

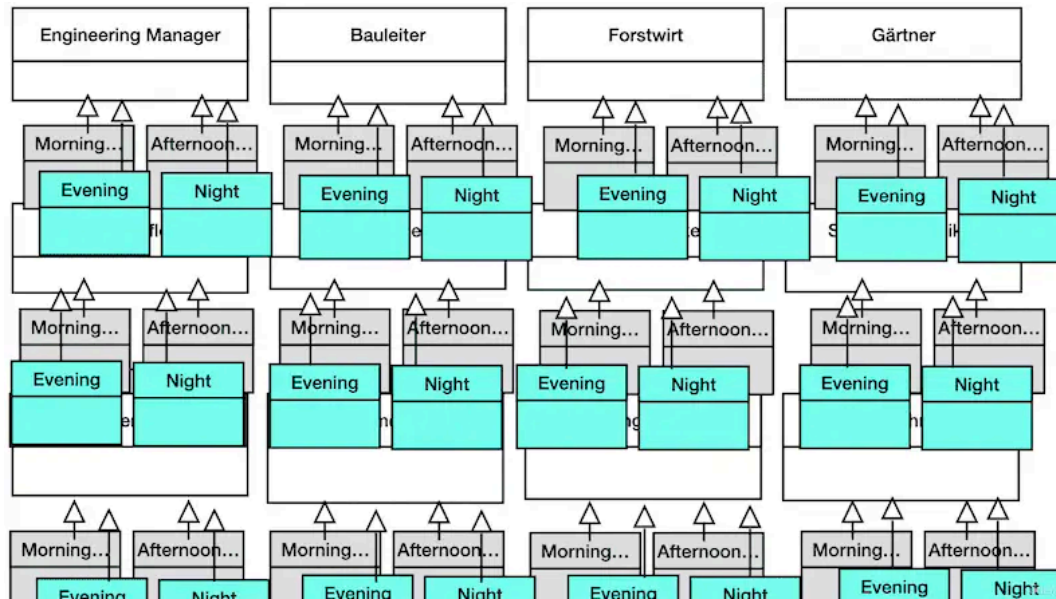
Engineering Manager	Bauleiter	Forstwirt	Gärtner
Fahrwegpfleger	Elektriker	Mechatroniker	Signalmechaniker
Gutachter Erdbau	Weichenmechaniker	Lokrangierführer	Lokführer

Explosion of classes

- We see the number of classes increases fast.



- What if we have more time-shift options?



IMPORTANT

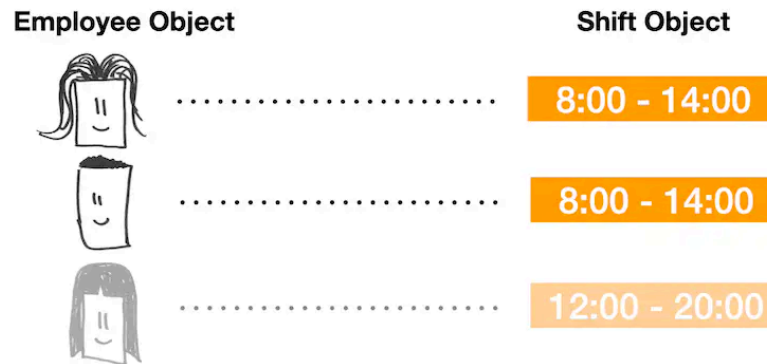
- We call this **Explosion of classes**.
- Most of the classes duplicate similar features.
- These are code smells and an indication of bad software design.

- **Warning:** Inheritance is not always the solution!

Explosion of classes
Duplicate code

Understand Business Logic

- Each employee has a specific time shift.
- From the business logic, we can see that time shift is another factor to manage.



- We could combine job role and shift into a single inheritance hierarchy (e.g., `MorningMechanic`, `AfternoonMechanic`, ...), but that recreates the same "explosion of classes" we just saw.
- The problem isn't inheritance itself — it's cramming two independently-varying concepts (job role and shift) into one hierarchy.

Composition as the Solution

- Job role and shift are two **independent dimensions of variation**.
- Instead of one combined hierarchy, we keep each dimension in its own small hierarchy and let **Employee** hold a **Shift** object — this is **composition**.

Composition Relationship

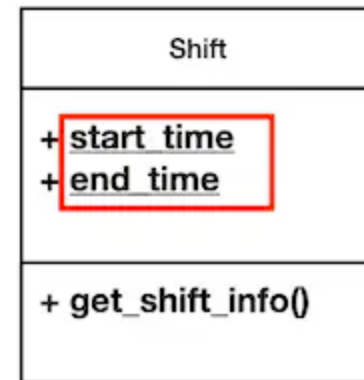
- We can make the Shift object a member of the Employee class using composition.
- This makes the relationship that Shift is a part of an Employee.
- Composition is a good fit when two concepts can change independently of each other (here: job role and shift).

IMPORTANT

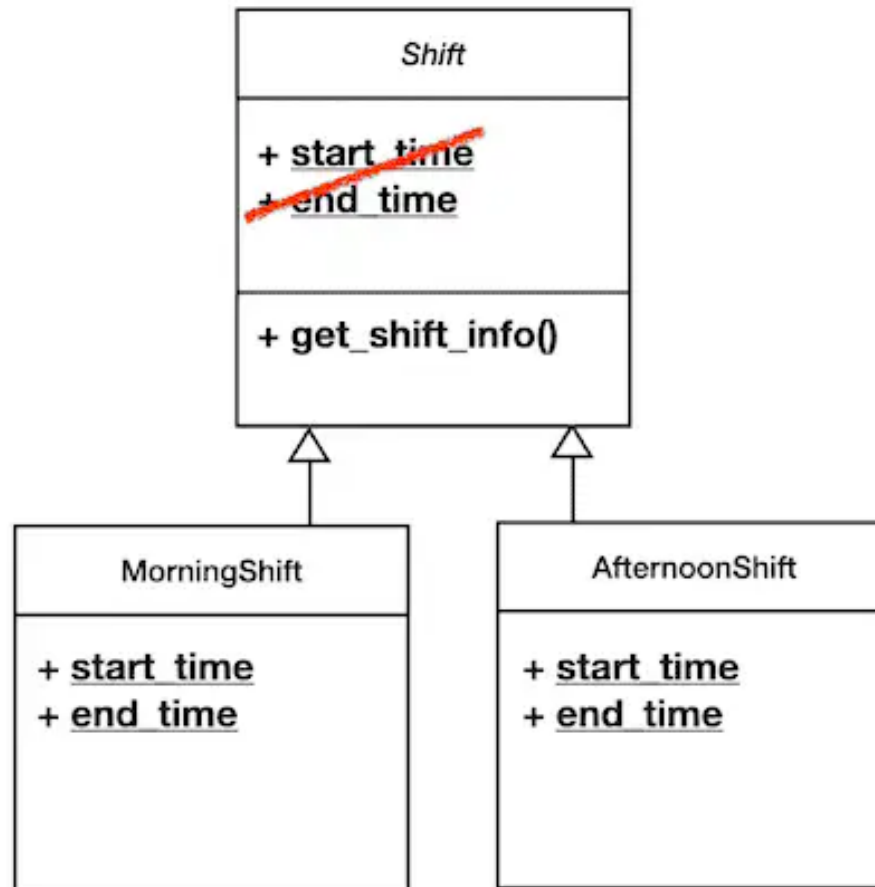
Shift Class Design

- Use inheritance for an **is-a** relationship.
- Use aggregation/composition for a **has-a** relationship.
- `Shift` is a part of `Employee`, not a type of `Employee`, so we use composition (or aggregation).

- So we need a new class.
- We can start creating the `Shift` class.



- Composition and inheritance are complementary.
- We still use inheritance *within* the shift hierarchy (e.g., `MorningShift`, `AfternoonShift`), and composition to combine it with the employee hierarchy.

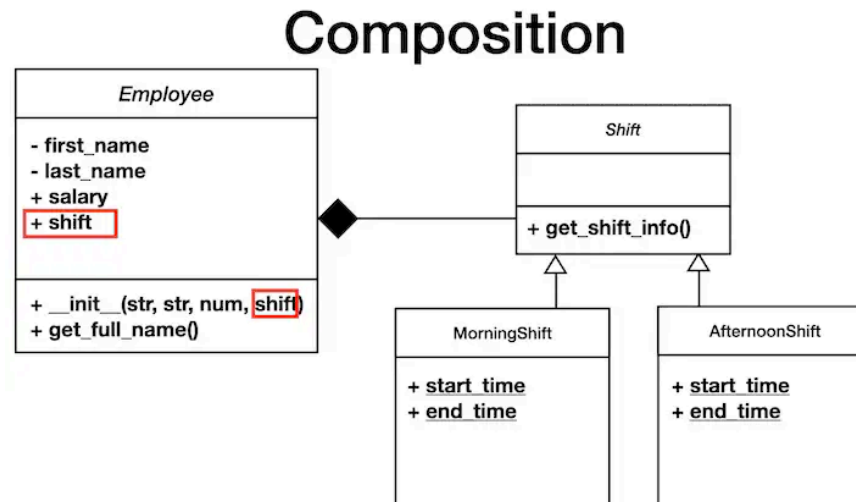


| Aggregation vs Composition

- **Aggregation:** the whole contains a part, but the part can exist independently.
- **Composition:** the part is essential to the whole and shares its lifecycle.
- Example: a wallet or phone is aggregated; a car is composed of an engine and body.

UML

- In UML, a **solid diamond** means composition.
- **Employee** owns its **Shift**, and the shift is part of the employee's lifecycle.
- This design intends each employee to have its own shift instance, not a shared one.



UML Details

- The Employee class must have only **one** Shift object, so there is no number or * in the connection.
- Notice that start and end time are all class variables that are shared by all objects.

Implementation

```
import datetime

class Shift:
    def get_shift_info(self):
        return f"{self.start_time:%H:%M} to {self.end_time:%H:%M}"

class MorningShift(Shift):
    start_time = datetime.time(8, 00)
    end_time = datetime.time(14, 00)

class AfternoonShift(Shift):
    start_time = datetime.time(12, 00)
    end_time = datetime.time(20, 00)
```

Python Pitfall

- Python uses `self` to express only the object member.
- However, start and end time are all **static** (class variables).
- So, there is no self in these fields.

Employee with Composition

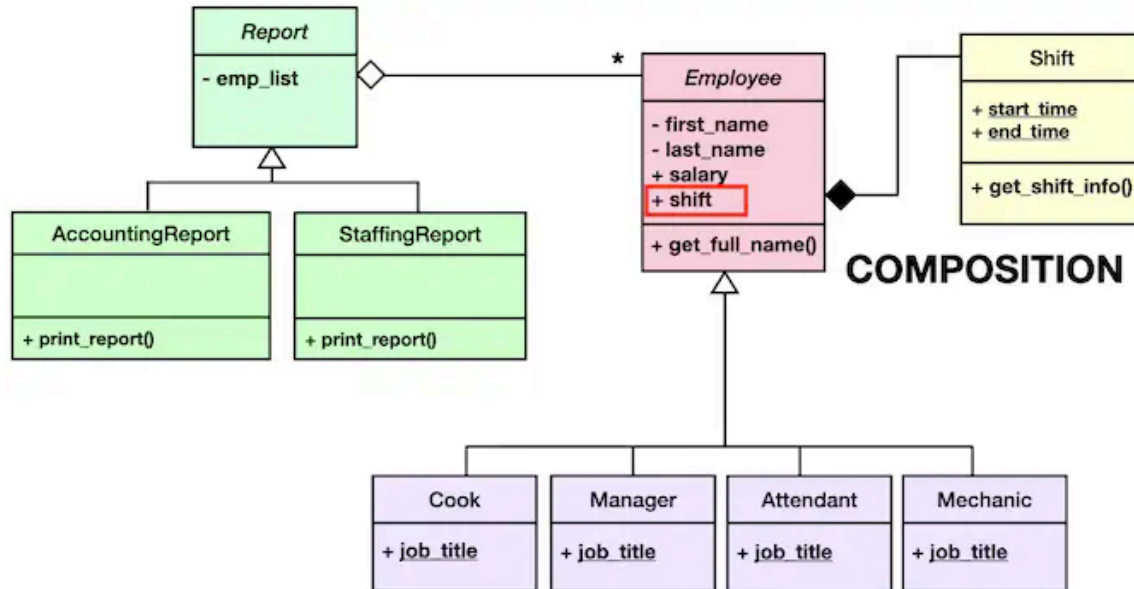
- The Employee class has an object shift as its fields to represent composition.

```
class Employee:
    def __init__(self, last_name, first_name, salary, shift):
        self._first_name = first_name
        self._last_name = last_name
        self.salary = salary
        self.shift = shift # shift object
```

Python Pitfall

- Python has no dedicated syntax or keyword for distinguishing UML aggregation from composition — both are implemented the same way, using an object reference stored as a member field.
- The difference (ownership, shared lifecycle) is a **design intent**, not something the language enforces.

The Final Design



Composition, Aggregation, and Inheritance

- The Report and Employee use the inheritance to remove duplication.
- The Employee and Shift use composition to avoid "explosion of classes."
- The Report uses Employee references gathered through aggregation; Report and Employee have separate responsibilities, supporting SRP.

Usage Example

```
from employee import Manager, Attendant, Mechanic
from shift import MorningShift, AfternoonShift

employees = [
    Manager("Schmidt", "Vera", 2000, MorningShift()),
    Attendant("Norris", "Chuck", 1800, MorningShift()),
    Mechanic("Rama", "Ringo", 1900, AfternoonShift()),
    Mechanic("Rainey", "Chuck", 1800, AfternoonShift()),
]
```

Easy Extension: Night Shift

- We have the software design to manage complexity from changes.
- What should we do to add the night shift?

FIRST NAME	LAST NAME	SHIFT
Vera	Schmidt	8:00 - 14:00
Chuck	Norris	8:00 - 14:00
Samantha	Carrington	12:00 - 20:00
Roberto	Jacketti	8:00 - 14:00
Dave	Dreißig	8:00 - 14:00
Tina	River	8:00 - 14:00
Ringo	Rama	12:00 - 20:00
Chuck	Rainey	14:00 - 22:00

Adding Night Shift

```
# shift.py
class NightShift(Shift):
    start_time = datetime.time(14, 00)
    end_time = datetime.time(22, 00)

# main.py
from shift import MorningShift, AfternoonShift, NightShift

employees = [
    # ... existing employees
    Mechanic("Rainey", "Chuck", 1800, NightShift()),
]
```

- Adding one class by sub-classing from the Shift solves the problem.
- With software design, it is easy to extend without modifying existing code!

Lessons Learned

- We may have an "*explosion of classes*" when we use inheritance.
- In this case, we can use *Composition*.

- **Design Principle:** Favor composition over inheritance when two concepts can change independently.
- **Benefits of Composition:**
 - Avoids class explosion
 - More flexible than inheritance
 - Easier to extend and maintain
 - Better separation of concerns